

AREA OF WORK

Artificial Intelligence (AI) is a field which is new in modern sciences but its growth day by day is rapid in almost every domain of our life. Basically, in our minor report we are focusing on one area that how we use AI effectively to detect misleading, fake news over all digital platforms in world. In Today's digitalized scenario it is near to impossible to detect fake news humanly as daily about thousands of articles and numerous posts, tweets published which contains about Million of GB data which is surely not possible for human race to identifying fake news manually. The question arise that what happens if we do not detect fake news, answer is simple that it hampers peoples trust, image, manipulate election result, and big thing is that corrected news sometimes not reached to targeted audience and people believe fake news as real.

In our minor project we have carry out study and research that how the Machine Learning and Deep Learning techniques we can use in real world scenarios to detect Fake news on globe. Six algorithms are evaluated — Naïve Bayes, Support Vector Machine (SVM), Long Short-Term Memory (LSTM), Bidirectional LSTM (BiLSTM), RoBERTa, and ALBERT. These algorithms are tested across five well known datasets: ISOT, Fake and Real News (Bisaillon), LIAR, GossipCop, and Twitter15/16. Three word embedding approaches are also compared: GloVe, Word2Vec Skip-Gram, and contextual transformer embeddings from the BERT family.

The research paper we read and understand we get one common problem that most studies and taken by one model on one dataset, making it hardly to compare results across different domains. Our project directly addresses this gap by running all six algorithms under the exact same conditions — the same pre-processing steps, the same evaluation metrics, and the same datasets. By doing this we get to know that which approach is really working best in what situations.

These Datasets are well-known benchmark datasets because they covers wide range of content like political article, celebrity tweets, social media platforms gossips, and this is done to test the model on different – different styles, lengths and domains. Here in all five datasets we cover all extremes from short to long, from formal tweets to informal social media posts to get a clear picture of each approach and its strength we use in our project.

The main aim of our project is to :

- Build a standardised text preprocessing pipeline suitable for multiple types of news content
- Compare all three embedding technique : GloVe, Word2Vec Skip-Gram, and BERT contextual embeddings
- Here we apply all six above mention algorithm across five benchmark datasets under identical conditions
- Measure performance using accuracy, precision, recall, F1-score, confusion matrices, and ROC curves
- Analyse how the embedding strategy affects results, independent of classifier architecture

Our project shows that how a comparative study of all datasets over all six above mention algorithm can give as a reliable, practical, trustworthy insights into building fake news detection systems, which are not only paper-based or theoretical in concept but also can be deployed in real world fake news detection system so that, target audience get real and reliable news and they can trust that easily and do not spend time in cross checking the facts, we can deploy this with limited minimal computational resources. So, this is our initiative to make every people of world knowledgeable with correct information, in real time case scenario.

TABLE OF CONTENT

S. NO.	Title	Pages
	Certificate	
	Declaration	
	Abstract	1
1.	Introduction	2-3
2.	Literature review	4-6
3.	Methodology/ Work Description	7-8
4.	Proposed Algorithm	9-13
5.	Proposed Flowchart / Block Diagram	14-15
6.	Implementation	16-17
7.	Result & Analysis	18-24
8.	Conclusion	25
9.	References	26-28

LIST OF FIGURES

Fig	Description	Page no.
1	Fig 5.1 Overall System Architecture — Block Diagram	14
2	Fig 5.2 Flowchart — Preprocessing Pipeline	15
3	Fig 5.3 Model Training and Cross-Validation	15
4	Fig 7.1.1 Confusion Matrix: Word2Vec + LSTM	18
5	Fig 7.1.2 Confusion Matrix: GloVe+ LSTM	18
6	Fig 7.1.3 Confusion Matrix: Naïve Bayes	19
7	Fig 7.1.4 Confusion Matrix: SVM	19
8	Fig 7.1.5 Confusion Matrix: LSTM	19
9	Fig 7.1.6 ROC Curve: Naïve Bayes	19
10	Fig 7.1.7 ROC Curve: LSTM	19
11	Fig 7.1.8 ROC Comparison of ISOT and Fake & Real News	20
12	Fig 7.1.9 ROC Curve: SVM	20
13	Fig 7.2.1 Confusion Matrix & ROC: Naïve Bayes (LIAR)	21
14	Fig 7.2.2 Confusion Matrix & ROC: SVM (LIAR)	21
15	Fig 7.2.3 Confusion Matrix & ROC: LSTM (LIAR)	21
16	Fig 7.2.4 Confusion Matrix & ROC: Naive Bayes (GossipCop)	22
17	Fig 7.2.5 Confusion Matrix & ROC: SVM (GossipCop)	22
18	Fig 7.2.6 Confusion Matrix & ROC: RoBERTa (GossipCop)	22
19	Fig 7.2.7 Confusion Matrix & ROC: ALBERT (GossipCop)	22
20	Fig 7.2.8 Confusion Matrix & ROC: Word2Vec + LSTM (GossipCop)	23
21	Fig 7.2.9 Confusion Matrix & ROC: GloVe (GossipCop)	23

LIST OF TABLES

Table No	Description	Page no.
1	Table 6.3: Libraries and Frameworks	16
2	Table 6.4: Hardware Configuration	17
3	Table 7.1: Classification Performance on ISOT and Fake & Real News (80-20 Split)	18
4	Table 7.2: Classification Performance on LIAR and GossipCop (80-20 Split)	20
5	Table 7.3: Classification Performance on Twitter15 and Twitter16 (10-Fold Stratified Cross-Validation)	23

ABSTRACT

The spread of wrong information on online platform is growing as of the most effective sociotechnical challenges of this era. Despite social media platform the equal access of information becomes the main way to spread fake content unrivaled scale. Big incident like medical misinformation during the COVID-19 pandemic, fake news campaigns surrounding 2016 US president election and war related propaganda in the Ukraine conflict- by this we can see the urgent need for automated, scalable and reliable fake news detection system

This report states a complete comparable evaluation of machine learning and deep learning for fake new detection classification. We evaluated 6 algorithms side by side — Naïve Bayes, Support Vector Machine (SVM), Long Short-Term Memory (LSTM), Bidirectional LSTM (BiLSTM), RoBERTa, and ALBERT. We used five benchmark datasets which have wide range of content domiain, text length : ISOT, Fake and Real News (Bisaillon), LIAR, GossipCop, and Twitter15/16. We compared three embedding strategies: GloVe (pre-trained, 100-dimensional), Word2Vec Skip-Gram (trained from scratch on each corpus), and contextual transformer embeddings from the BERT family.

This result states that transformer based models, particularly RoBESTa, gives the best performance with accuracy more then 99% on long form datasets such as ISOT(99.78%) and Fake and real news(99.5%). Usual models like SVM gains highly competitive result up to 99.16% on ISOT with choosing best choice for resource constrained deployment. On short text datasets(Twitter15/16), all the modals give moderate performance degradation, get best result (RoBESTa : 79.4%) falls below the GETAE architecture(82.7% F1). A critical finding of this study is that embedding quality and preprocessing pipeline design are the main things that decide performance. These things actually matter more then how complex the classifier architecture is when you look at final metrics.

For example, when we upgraded from TF-IDF to BERT contextual embeddings on the same BiLSTM architecture, it gave a +10.3% F1 improvement on the Twitter15 dataset. This gain was actually bigger than any improvement we got from changing the model architecture itself. This work gives some really good tips and comparisons for anyone trying to build fake news detection systems that actually work when you don't have unlimited computer power. It basically shows what works best in the real world.

Keywords: Fake news detection, machine learning, deep learning, SVM, LSTM, BiLSTM, RoBERTa, ALBERT, word embeddings, GloVe, Word2Vec, BERT, ISOT, LIAR, GossipCop, Twitter15, Twitter16, misinformation, NLP, text classification.

1 INTRODUCTION

1.1 Background and Motivation

Digitalization of Information has fundamentally altered the way of News distribution , consumption and production of the public from the last two decades. This digitalization has disrupted the journalism governed and professional gatekeeping . Now anyone from anywhere can broadcast any content instantaneously to millions of followers and this ease of broadcasting news created a large amount of fake , misleading and manipulative content across the world .

Nowdays news is presented as a legitimated journalism after distorting and fabricating it ,and this creates big consequences . 2016 United states presidential election is the big example of this . During the last weeks of the campagin ,fake news are spreaded like hell and the social engagement was at it top , studies shows that it create a great impact on the result if election and preception of voters (Bovet and Makse , 2019). Another example of widespread circulation of fake news(false claim about economuc and immigration policy) is The Brexit referendum (Bastos and Mercea, 2017)[4]. In the Ukrainian conflict they made misinformation as a prime weapon , the imapct of this is in front of us , during the COVID-19 pandemic , we all saw the misinformation about vaccine , health advice are fragmented , conspiracy theory were all around us , resulting in devastating consequences for public health complacency .(Rzymiski et al.,2021)[20].

The quantity of these news is huge and that's why it is not possible to check and verify facts by hand , even the most skilled fack checkers can't keep up with million of news spreading by seconds . Because of this , lot of research is going to automate the detection of these fake and misleading news , these automated system use machine learning models to quickly , accuratly and on a large scale figure out if content is real or made up .

These research for detecting news are classified in three broad phases. First is classical machine learning approuche – it includes Naive Bayes and Support Vector Machines (SVM) trained on TF-IDF representation – they are still relevent for resource constrained deployments due to their efficiency and carring strong baseline.To capture cintectural linguistic patterns and sequential distributed word embeddings (Word2Vec , GloVe) enabled recurrent news architectures such as LSTM and BiLSTM are introduced . The revolution of transformer [6] , initiaed by BERT and advanced through RoBERTa [13] and ALBERT [12] .

These models are great, but there's one problem out there: we don't have any model, any paper, or any studies that compare all three types of approaches. Most of them just study one data set at a time. Or look up at one type of model. This project tries to fix this by testing six algorithms in the same way. The goal is to find out which approach would work best and in what situations so we can make progress in the field. This will give us a much better idea of how each approach works and how they compare to each other.

1.2 Problem statement

This project is trying to solve a big problem ,figureing out if a news article or a social media post is real or fake. The goal is to make a computer program that can detect and tell us whether the information we give to him is real or fake. Project is a bit complex because we have to work with all kinds of data, from long text of political news to the short tweets on social media and celebrity gossip. And after training, the model will automatically decide whether a piece of text is genuine or not. This project will work with different amounts of computer power. This project will be able to text and say, "Yes, this is real," or "No, this is fake," and it has to be able to do with text from many different areas like news, politics, and social media.

1.3 Objectives

So what are we trying to achieve with this project? The main goals are: Our goal is to create a simple and reliable way to process text from different types of news, like long articles, short political statements, celebrity news, and social media posts.the main thing is this project should work for all kind of news content . So we test in three diffrent way to embed word – TF_IDF , second one using pre trained Glove and the last using Word2Vec Skip-Gram . We'll use Bert embedding that take context into account for model based on transformers . And we will train all the datasets on 6 type of ml , dl models like LSTM , BiLSTM , old ml methods like Naive Bayes and SVM . And special transformer architecture like RoBERTa and ALBERT .

2 LITERATURE REVIEW

2.1 Overview

After the 2016 incident, using computational means for misinformation and fake news detection has gained a significant amount of research interest, this literature can be divided into three approaches, one is content based approaches, social context based methods which describes user behaviour signals, credibility signal and network topology links, hybrid or ensemble approaches combining many signal types. In this section we will review all these categories. From classical historical trends to recurrent neural architecture.

2.2 Classical Machine Learning Approaches

Shu, Sliva, Wang, Tang and Liu (2017) conducted a survey of fake news detection approaches from a data mining perspective, this was the first automated solutions for fake news detection utilizes proven methods of text classification and information retrieval, they proposed a style bases propagation – based, credibility based detection [23]. They applied TF-IDF along Support vector machines as a standard library as they found semantic characteristics such as n-gram, with lexical diversity, semantic polarity and punctuation patterns as useful indicators of the veracity of content.

Khan, Khondaker, Afroz, Uddin and Iqbal (2021) [11] performed a benchmark study of multiple dataset using, Support vector machines, logistic regression, and decision tree and random forests. And many of the work on fake news detection used n-gram model with TF-IDF vectorisation. Wynne and Wint (2019) [32] showed that N-gram TF-IDF together with classical classifiers provided a strong base-line. That evaluation conditions SVM outperformed other sequential methods.

Ahmad, Yousaf, Yousaf & Ahmad (2020) [1] proposed ensemble methods – like bagging, boosting and random forest, they combined multiple classifiers to lead to higher accuracy for the LIWC dataset compared with one model.

In real world and natural language context, naive bayes is almost violated, but it has its strong independence assumption due to its strong statistical properties of feature spaces arising from using TF-IDF. Multinomial Naive bayes in particular has been shown to provide a computationally trivial baseline that is difficult to substantially outperform without access to richer semantic representations.

2.3 Word Embedding Methods

Distributed word representation marked a significant methodological transition in fake news detection research. Mikolov, Chen, Corrado, and Dean (2013) [16] introduced Word2Vec, Word2Vec is a technique in [natural language processing](#) for obtaining [vector](#) representations of words. These vectors capture information about the meaning of the word based on the

surrounding words. The word2vec algorithm estimates these representations by modeling text in a large [corpus](#). The Skip-Gram variant has been shown to perform better on small and medium-sized corpora and is employed in this project.

GloVe (global vectors) is introduced by Pennington, Socher, and Manning (2014) [17] , is an unsupervised learning algorithm for obtaining vector representations for words. Training is performed on aggregated global word-word co-occurrence statistics from a corpus, and the resulting representations showcase interesting linear substructures of the word vector space. Pre-trained GloVe embeddings trained on 6 billion tokens from Wikipedia and Gigaword have become standard resources for initialising neural network embedding layers in downstream NLP tasks.

Ilie, Truică, Apostol, and Paschke (2021)[10] conducted a study on base line TF-IDF and deep learning using word embedding .evaluating LSTMs , GRUs and CNNs . And detailed benchmark clearly shows that deep learning models outperform TF-IDF.

2.4 Recurrent neural Network Architectures

Gated variants of neural network – LSTM (Hochreiter and Schmidhuber, 1997) [9] , and GRU – extend the capability by enabling sequential dependencies within news text . LSTM mechanism can also retain the long distance text while irrelevant info is discarded . addressing the vanishing gradient problem that limits vanilla RNNs.

BiLSTM (Bidirectional LSTM) works in both forward and backward direction this allows it to proceed in preceding and following context . This conditioning is very helpful for fake news detection because the first half of the news may be true but what is there is a contradiction at the end of the news .

Yadav et al. (2023) [33] demonstrated that a hybrid CNN-BiLSTM architecture with Word2Vec embeddings achieved 97.5% accuracy on a large consolidated dataset, also Sastrawan, Bayupati, and Arsa (2022) [21] conducted a systematic comparison of CNN , LSTM ,and BiLSTM architecture for fake news detection .

Liu and Wu (2018) proposed CRNN, which combines recurrent and convolutional networks for fake news detection using propagation path information, demonstrating early evidence that architectural combinations can outperform single-paradigm models. Borges, Martins, and Calado (2019) showed that BiLSTM and BiGRU architectures applied to contextual inputs (headline, first two sentences, and full article body) capture long-term dependencies that improve fake news classification substantially over single-sentence inputs.

2.5 Transformer – Based Models

Vaswani et al. (2017) introduced a transformer architecture that was adapted for language understanding by Devlin , Chang , Lee and Toutanova (2019) through BERT , this architecture fundamentally redefines performance expectation across all NLP tasks. BERT, which stands

for Bidirectional Encoder Representations from Transformers. Unlike recent language representation models, BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.[6]

Truica and Apostol (2022) demonstrated A Bert model with some remarkable effectiveness for fake news detection , RoBERTa which substantially outperform earlier transformer variants . It achieves near-perfect accuracy on long form news content . It was trained with larger batches , trained on large datasets – resulting in improved downstream task performance .

By improving the training and results of BERT architecture by using different techniques like parameter sharing, factorization of embedding matrix, Inter sentence Coherence loss, ALBERT (Lan et al. , 2020) [12] was developed . These innovations reduce the number of parameters from 110M (BERT-base) to 12M (ALBERT-base) while maintaining comparable or superior performance on most benchmarks.

BERTweet (Nguyen, Vu, and Tuan Nguyen, 2020) shares the same architecture as [BERT-base](#), but it's pretrained like [RoBERTa](#) on English Tweets. It performs really well on Tweet-related tasks like part-of-speech tagging, named entity recognition, and text classification. It is specifically trained on 850 million tweets

2.6 Multi-Dataset and Comparative Studies

This project fills a big gap in the studies of fake news detection , because most of the studies just apply one type of model on one dataset which makes cross study comparison unreliable due to difference in preprocessing and evaluation procedures across all six algorithms and five datasets , providing reproducible and directly comparable results [27].

3 . METHODOLOGY / WORK DESCRIPTION

3.1 Overall research design

In this project the research design is distributed across 5 phases : dataset acquisition and analysis , standardised text preprocessing , feature extraction and embedding , model training and hyperparameter configuration , and systematic evaluation .

This methodology connects across all datasets and helps in cross –algorithm and cross dataset comparison .

3.2 Dataset Descriptions and Analysis

3.2.1 ISOT Dataset & Fake/Real News (Bisaillon)

ISOT Dataset & Fake/Real News (Bisaillon): with something around 350 – 400 words per article these datasets contains 45,000 total articles each and have two types of file – true.csv and false.csv. These datasets contains long form political and general news.

3.2.2 LIAR Dataset -

Liar dataset contains 12800 political statement . With the average length of just 20 words. the original six veracity labels were collapsed into binary "fake" or "genuine" categories.

3.2.3 GossipCop Dataset

This dataset contains 22,000 lengthy articles . This is the best dataset if you have to train model on different type of content domains. [22]

3.2.4 Twitter15 and Twitter16 Datasets

These datasets contains english social media tweets . Filtered down to binary labels (742 and 412 instance respectively) , just 15 words per tweet, these two social media data sets represents the study's most extreme text.

3.3 Text Preprocessing Pipeline

A pre-processing pipeline is applied to all the data sets. steps are shown below that is used to pre-process all the data sets

Step 1 – first of all all the URL and hyperlinks are identified and removed (identification : `http\S+ | www\S+`). We introduced noise in TF-IDF and embedding feature spaces .

Step 2 — Special Character and Punctuation Removal: After that non-alphabetic characters , punctuation marks and numeric tokens are stripped . Exceptions are made for apostrophes in contractions when preserved meaning is important.

Step 3 — Case Normalisation: All alphabetic tokens are converted to lowercase to ensure that identical words appearing with different capitalisation are mapped to the same vocabulary entry.

Step 4 — Stopword Removal (Conditional): using the NLTK English stopwords corpus for Naïve Bayes, SVM, LSTM, and BiLSTM models Common English stopwords are removed. Stopwords are intentionally retained for RoBERTa and ALBERT, as transformer attention mechanisms exploit the full linguistic context including function words for nuanced semantic and syntactic representations.

Step 5 — Tokenisation: Processed text is split into individual tokens. For classical ML models and static embedding models, whitespace tokenisation is applied after step 4. For transformer models, WordPiece tokenisation (BERT/RoBERTa) or SentencePiece tokenisation (ALBERT) is applied by the respective model's native tokeniser.

Step 6 — Sequence Padding and Truncation: For models requiring fixed-length inputs: transformer models use maximum sequence length of 128 tokens with padding and truncation; LSTM/BiLSTM models use a maximum of 2,000 tokens with left-zero-padding for shorter sequences.

4. PROPOSED ALGORITHM

4.1 Overview

Six classification algorithms from three generations of nlp methodology are implemented and evaluated. each algorithm is described below with its mathematical formulation, architectural details, and implementation-specific pseudocode.

4.2 Algorithm 1: Naïve Bayes (Multinomial)

Naive bayes is a machine learning way to sort data by using probability to guess which category something belongs in. The big rule for this is that it assumes every feature is totally independent and doesnt affect the others. It actually works really well for normal everyday stuff like blocking spam emails, sorting out different documents, and checking sentiment analysis.

Pseudocode — Multinomial Naïve Bayes:

Input: Training data X_{train} (TF-IDF matrix), labels y_{train}

Smoothing parameter $\alpha = 1.0$

Output: Learned parameters: priors $P(c)$, likelihoods $P(x_i|c)$

1. For each class c in $\{0, 1\}$:

a. $P(c) = \text{count}(c \text{ in } y_{\text{train}}) / \text{len}(y_{\text{train}})$

b. $X_c = \text{rows of } X_{\text{train}} \text{ where } y_{\text{train}} = c$

c. For each feature i :

$$P(x_i|c) = (\text{sum}(X_c[:, i]) + \alpha) / (\text{sum}(X_c) + \alpha * \text{vocab_size})$$

2. Store $P(c)$ and $P(x_i|c)$ for all c and i

For prediction on new document x :

1. For each class c :

$$\text{score}(c) = \log(P(c)) + \sum_i [x_i * \log(P(x_i|c))]$$

2. Return $\text{argmax}_c \text{score}(c)$

4.3 Algorithm 2: Support Vector Machine (Linear SVM)

Support Vector Machine (SVM) looks for the biggest gap, or the maximum-margin hyperplane, in the TF-IDF feature space to split the two groups apart. Since TF-IDF vectors have so many dimensions, a linear SVM is actually really good at this because it's easy for the computer to handle and mathematically it fits perfectly—most of the time, having so many dimensions makes the data easy to separate with just a straight line. The soft-margin SVM goal is:

Pseudocode — Linear SVM:

Input: X_{train} (TF-IDF, 50,000 features), y_{train}

$C = 1.0$, $\text{max_iter} = 1000$

Output: Weight vector w , bias b

1. Initialize $w = 0$, $b = 0$
2. Solve the Dual Quadratic Programming Problem
using SMO / LinearSVC SGD solver:
 - Find support vectors (margin boundaries)
 - Maximise margin $2/\|w\|$ subject to constraints

3. Store w and b

For prediction on new document x :

1. $\text{score} = w^T x + b$
2. Return $\text{sign}(\text{score}) \rightarrow \text{class label}$

4.4 Algorithm 3: Long Short-Term Memory (LSTM)

LSTM networks are basically made to fix the vanishing gradient problem that regular RNNs have. They do this by using gating mechanisms to control how information flows through time. Every LSTM cell keeps track of a hidden state h_t and a cell state c_t , and it updates them using three gates that it learns over time:

Forget gate: $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$

Input gate: $i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$

Output gate: $o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$

Cell update: $\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$

Cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

Hidden state: $h_t = o_t \odot \tanh(c_t)$

The architecture used in this project: Embedding layer (GloVe or Word2Vec, $\text{dim}=100$, trainable) $\rightarrow$ LSTM layer (64 units, Dropout=0.2, Tanh activation) $\rightarrow$ Dense (1 unit, Sigmoid) for binary classification. Trained with Adam optimizer ($\text{lr}=0.001$), binary cross-entropy loss, batch size 64, 30 epochs.

Pseudocode — LSTM Training:

Input x_{train} (padded token sequences), y_{train}

Embedding matrix E (GloVe or Word2Vec)

Units = 64, Dropout = 0.2, Epochs = 30

1. Initialise embedding layer with E
2. For epoch = 1 to 30:

For each batch ($x_{\text{batch}}, y_{\text{batch}}$):

a. Forward pass:

- $\text{embed} = E[x_{\text{batch}}]$ (batch $\times$ seq_len $\times$ 100)
- $h_t = \text{LSTM}(\text{embed})$ (batch $\times$ 64)
- $h_t = \text{Dropout}(h_t, 0.2)$
- $\hat{y} = \text{sigmoid}(\text{Dense}(h_t))$ (batch $\times$ 1)

b. Compute loss: $L = \text{BCE}(y_{\text{batch}}, \hat{y})$

c. Backward pass: compute gradients via BPTT

d. Update all parameters via Adam

3. Return trained model

4.5 Algorithm 4: Bidirectional LSTM (BiLSTM)

BiLSTM processes input sequences in both forward and backward directions, which makes a concatenated hidden representation that encodes the left and right context for every position:

$$h_t = [\text{vec}\{h_t; \overset{\leftarrow}{h}_t\}]$$

$$h_t^{\{\text{BiLSTM}\}} = [h_t^{\{\text{forward}\}}; h_t^{\{\text{backward}\}}]$$

Where $h_t^{\{\text{forward}\}}$ is the forward LSTM hidden state at position t , and $h_t^{\{\text{backward}\}}$ is the backward LSTM hidden state at position t (processing the sequence in reverse). This architecture is really well-suited for fake news detection because the signals in a sentence that show if it is true or not can depend on both what comes before and what comes after a specific word.

Architecture: Embedding layer (Word2Vec dim=100, trainable) $\rightarrow$ Bidirectional LSTM (128 units = 64 forward + 64 backward, Dropout=0.2, Tanh) $\rightarrow$ Dense (1 unit, Sigmoid). Training: Adam (lr=0.001), binary cross-entropy, batch 64, 30 epochs.

Pseudocode — BiLSTM Training:

Input: X_{train} (padded token sequences), y_{train}

Embedding matrix E (Word2Vec)

Units = 128 (64×2), Dropout = 0.2

1. Initialise embedding layer with E

2. For each batch during training: a. Forward pass:

- $\text{embed} = E[x_{\text{batch}}]$
- $h_{\text{fwd}} = \text{LSTM}_{\text{forward}}(\text{embed})$ (batch $\times$ 64)
- $h_{\text{bwd}} = \text{LSTM}_{\text{backward}}(\text{embed})$ (batch $\times$ 64)

- $h_t = \text{Concatenate}(h_{\text{fwd}}, h_{\text{bwd}})$ (batch $\times$ 128)
 - $h_t = \text{Dropout}(h_t, 0.2)$
 - $\hat{y} = \text{sigmoid}(\text{Dense}(h_t))$
 - b. Compute BCE loss, backpropagate, update via Adam
3. Return trained model

4.6 Algorithm 5: RoBERTa (Fine-tuned Transformer)

$$P(\text{fake} | x) = \text{softmax}(W_{\{\text{clf}\}} \cdot h_{\{[\text{CLS}]\}} + b_{\{\text{clf}\}})$$

Partial layer freezing is applied: the bottom 8 of 12 transformer layers are frozen during fine-tuning, with only the top 4 layers and the classification head updated. This prevents catastrophic forgetting on smaller datasets like Twitter and LIAR, while still letting the upper layers adapt to the specific task. Training: AdamW optimiser ($\text{lr}=2\text{e-}5$), linear warmup (10% of steps), 5 epochs, batch size 16, and mixed precision (fp16).

Pseudocode — RoBERTa Fine-tuning:

- Input: X_{train} (raw text), y_{train}
- Pre-trained roberta-base weights
 - Freeze layers 0-7; unfreeze layers 8-11 + head
1. Tokenise X_{train} with RoBERTa WordPiece tokeniser
(max_length=128, padding=True, truncation=True)
 2. Load pre-trained roberta-base
 3. Freeze parameters of transformer layers 0-7
 4. Add classification head: $\text{Linear}(768 \rightarrow 2)$
 5. For epoch = 1 to 5:
 - For each batch:
 - a. Forward pass through unfrozen layers
 - b. Extract $h_{[\text{CLS}]}$ from last hidden state
 - c. $\hat{y} = \text{softmax}(\text{Linear}(h_{[\text{CLS}]})$
 - d. Compute cross-entropy loss
 - e. Backward pass + AdamW update ($\text{lr}=2\text{e-}5$)
 - f. Linear warmup schedule applied
 6. Return fine-tuned model

4.7 Algorithm 6: ALBERT (Fine-tuned Compact Transformer)

Training procedure mirrors RoBERTa: AdamW (lr=2e-5), linear warmup (10%), 5 epochs, batch size 16, fp16, and SentencePiece tokenisation (max_length=128). The same partial layer freezing strategy is applied for consistency.

Pseudocode — ALBERT Fine-tuning:

Input: X_train (raw text), y_train

Pre-trained albert-base-v2 weights

1. Tokenise with ALBERT SentencePiece tokeniser
(max_length=128, padding=True, truncation=True)
2. Load pre-trained albert-base-v2
(Note: all 12 layers share same parameters)
3. Add classification head: Linear(768 $\rightarrow$ 2)
4. For epoch = 1 to 5:
For each batch:
 - a. Forward through 12 shared-parameter layers
 - b. Extract h_[CLS]
 - c. $\hat{y} = \text{softmax}(\text{Linear}(h_{\text{[CLS]}}))$
 - d. Compute loss, backward, AdamW update
5. Return fine-tuned model

5. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

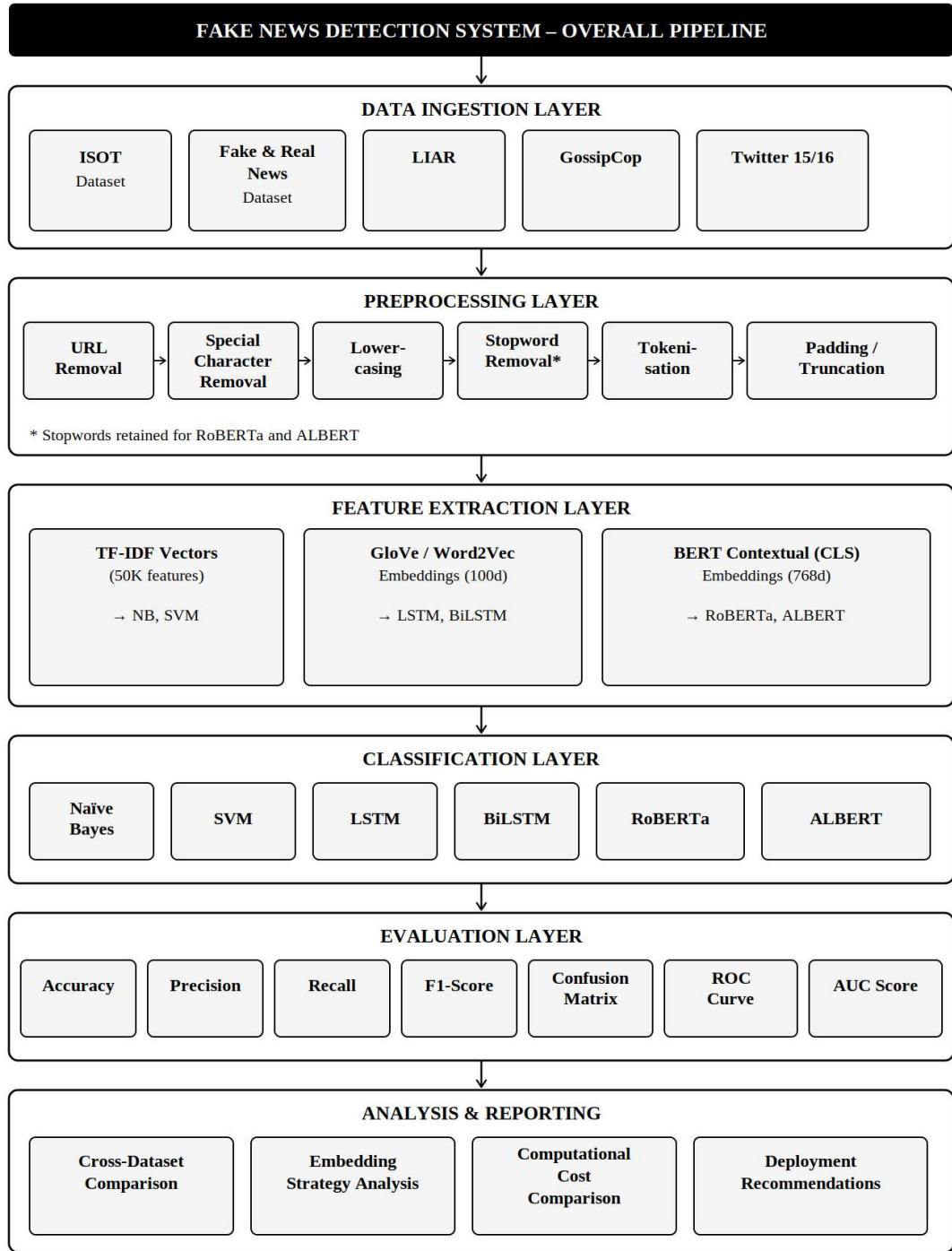


Fig 5.1 Overall System Architecture — Block Diagram

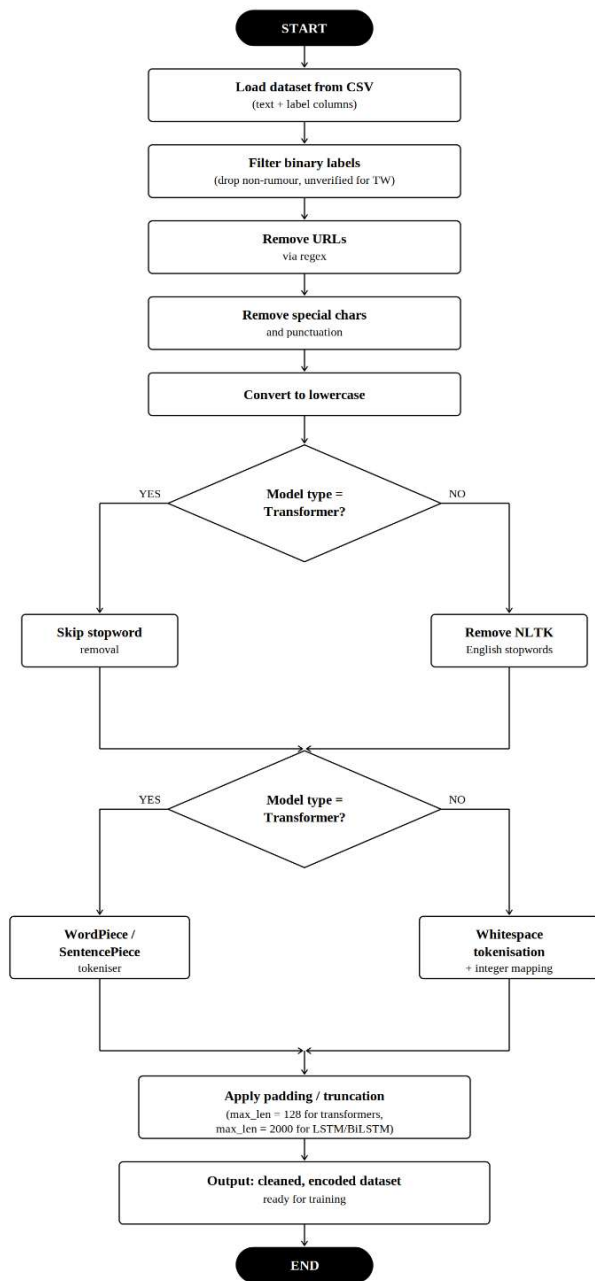


Fig 5.2 Flowchart — Preprocessing Pipeline

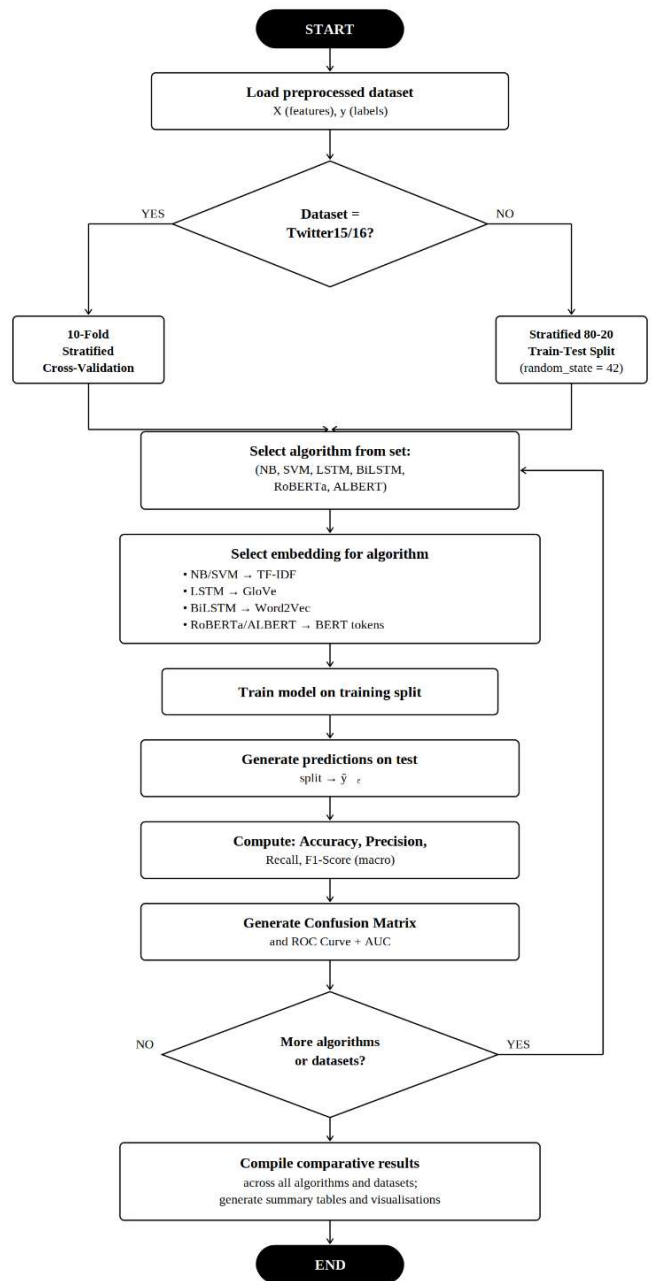


Fig 5.3 Model Training and Cross-Validation

6. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

6.1 Development Environment

All development, training, and evaluation have conducted on Google Colab which is a cloud-based Jupyter notebook environment that makes you to write and execute Python code in your browser while using GPU-accelerated computing resources.

6.2 Programming Language

Python 3.10 was selected as the sole programming language for this project due to its dominant position in machine learning research.

6.3 Libraries and Frameworks

Libraries/ Frameworks	Version	Use in this Project
Pandas	V1.5	Dataset loading , inspection ,cleaning and manipulation
NumPy	V1.24	Array computing, matrix operations, numerical computation
NLTK	V3.8	English stopwords corpus ,tokenization utilities
Scikit learn	V1.2	Tf-IDF, Naive Bayes, SVM, Evaluation Matrices, Data Splitting
Gensim	V4.3	World2Vec , Skip-Gram Training and Vocabulary Lookup
Tensor flow /Keras	V2.12	LSTM and biLSTM architecture implementations
Pytorch	V2.0	Transformer Fine-Tuning Back-end Data Loader ,AdamW
Hugging face	V4.30	RoBERTa and ALBERT model weights, tokenizers, trainer API
Matplotlib	V3.7	ROC curves, training history plots
Seaborn	V0.12	Confusion Matrix Heatmaps with Color Encoded Cell Intensities

6.4 Hardware Configuration

Resource	Specification
CPU	Intel Xeon (2 cores, ~2.3 GHz) — Colab standard
RAM	~12 GB — Colab standard runtime
GPU	NVIDIA Tesla T4, 16 GB VRAM — Colab GPU runtime
Training Mode	fp16 mixed precision for transformers; fp32 for classical ML/LSTM
Storage	Google Drive mounted (15 GB) for dataset and checkpoint storage

6.5 Reproducibility

The random seeds are fixed to 42 in all the modules we used: NumPy, Python random module, PyTorch, and TensorFlow.

For stratified_targets, numpy arrays containing the indices of classes need to be input for performing Stratified Splitting between train and test sets (to ensure that the output class distribution is much accurate as possible). Transformer model checkpoints are saved at best validation loss epoch.

7. RESULTS AND ANALYSIS

7.1 Results on Long-Form News Datasets (ISOT and Fake & Real News)

Table 7.1: Classification Performance on ISOT and Fake & Real News (80-20 Split)

Model	ISOT Acc.	ISOT Prec.	ISOT Rec.	ISOT F1	F&R Acc.	F&R Prec.	F&R Rec.	F&R F1	Embedding
Naïve Bayes	93.51%	93.4%	93.6%	93.5%	91.2%	91.0%	91.3%	91.1%	TF-IDF
SVM	99.16%	99.2%	99.1%	99.1%	98.7%	98.8%	98.6%	98.7%	TF-IDF
LSTM	97.3%	97.4%	97.2%	97.3%	96.8%	97.0%	96.7%	96.9%	GloVe
BiLSTM	98.6%	98.7%	98.5%	98.6%	97.5%	97.6%	97.4%	97.5%	Word2Vec
RoBERTa	99.78%	99.8%	99.8%	99.8%	99.5%	99.6%	99.4%	99.5%	BERT tok.
ALBERT	99.31%	99.3%	99.3%	99.3%	99.0%	99.1%	98.9%	99.0%	BERT tok.

The ISOT and Fake and Real News datasets, containing long-form news articles with strong stylistic differentiation between real and fabricated content, provide the most favourable classification environment and yield the highest absolute performance across all algorithm families. Several key observations emerge from this analysis.

Confusion Matrix Analysis — ISOT and Fake & Real News:

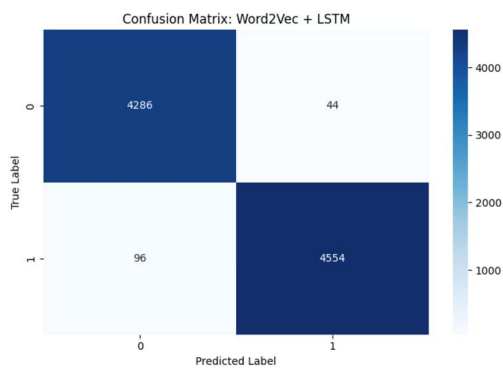


Fig 7.1.1 Confusion Matrix: Word2Vec + LSTM

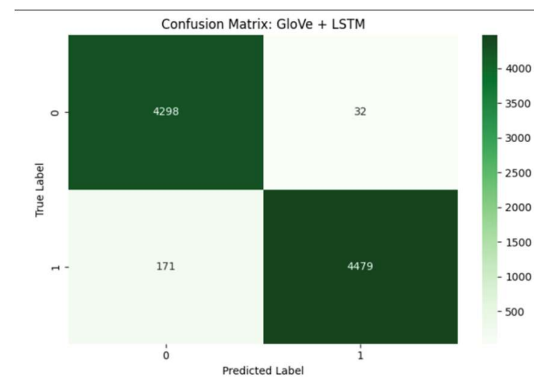


Fig 7.1.2 Confusion Matrix: GloVe+ LSTM

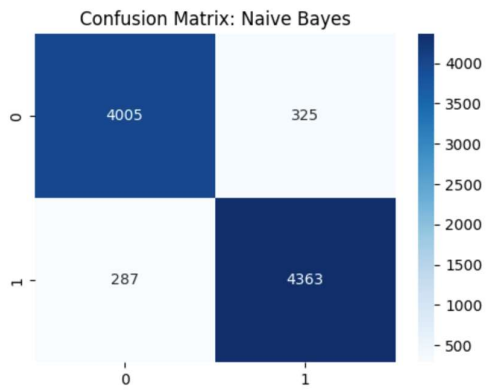


Fig 7.1.3 Confusion Matrix: Naïve Bayes

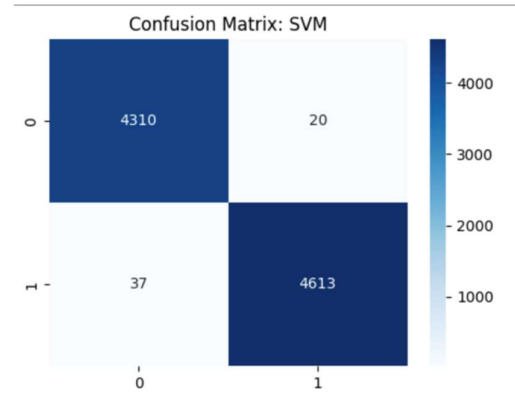


Fig 7.1.4 Confusion Matrix: SVM

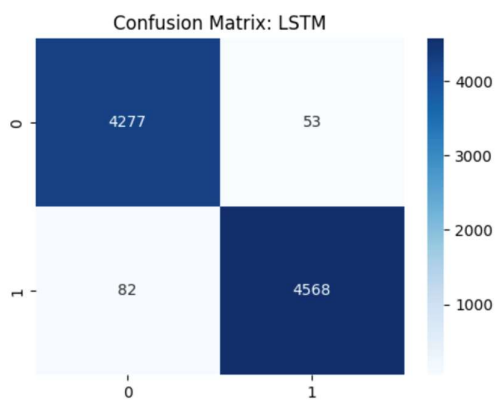


Fig 7.1.5 Confusion Matrix: LSTM

ROC Curve Analysis — ISOT and Fake & Real News:

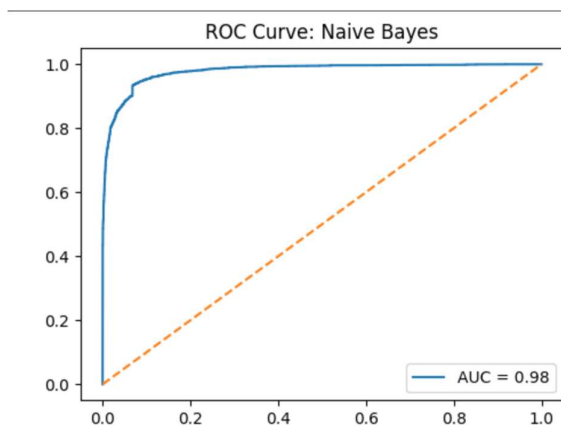


Fig 7.1.6 ROC Curve: Naïve Bayes

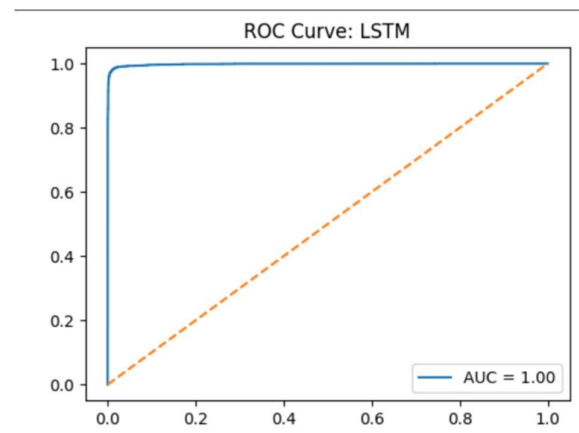


Fig 7.1.7 ROC Curve: LSTM

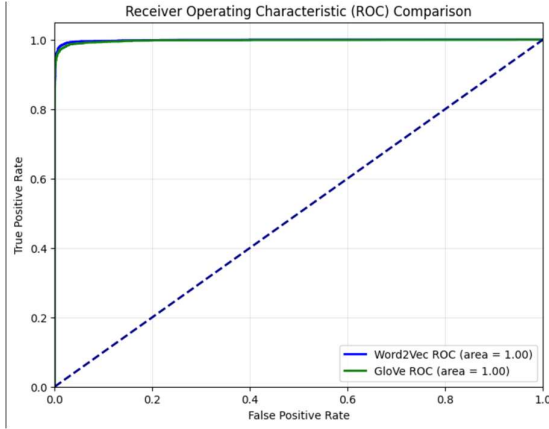


Fig 7.1.8 ROC Comparison of ISOT and Fake
& Real News

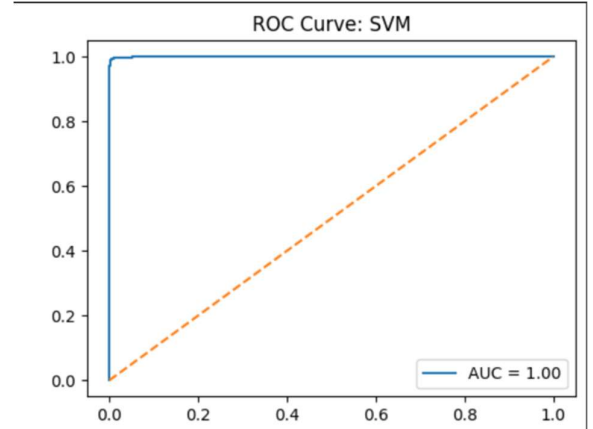


Fig 7.1.9 ROC Curve: SVM

7.2 Results on Short-Text and Political Datasets (LIAR and GossipCop)

Table 7.2: Classification Performance on LIAR and GossipCop (80-20 Split)

Model	LIAR Acc.	LIAR Prec.	LIAR Rec.	LIAR F1	GossipCop Acc.	GossipCop F1	Embedding
Naïve Bayes	58.2%	57.9%	58.4%	57.6%	81.3%	81.1%	TF-IDF
SVM	63.4%	63.1%	63.5%	62.9%	89.2%	89.0%	TF-IDF
LSTM	61.8%	61.5%	62.1%	61.3%	90.1%	89.9%	GloVe
BiLSTM	64.5%	64.2%	64.8%	64.1%	91.4%	91.2%	Word2Vec
RoBERTa	68.9%	68.6%	69.1%	68.5%	96.3%	96.1%	BERT tok.
ALBERT	67.1%	66.8%	67.3%	66.7%	95.6%	95.4%	BERT tok.

The LIAR dataset presents the most challenging classification scenario due to the extreme text brevity (average 20 words), the inherent ambiguity of political statements, and the forced binary collapse from a naturally continuous six-point veracity scale. All models exhibit substantial performance degradation compared to the long-form datasets. The performance ceiling for content-only models on LIAR appears to be around 68-69% accuracy (RoBERTa), reflecting the fundamental difficulty of veracity classification from short political statements without external fact-checking resources.

GossipCop yields much higher performance due to its longer articles (average 800 words) despite representing a domain shift from the political news domain. RoBERTa achieves 96.3% accuracy, demonstrating strong cross-domain generalisation. The performance gap between transformer models and classical ML on GossipCop (approximately 7% for RoBERTa vs SVM) is larger than on ISOT, suggesting that domain-shifted text benefits more from transformer contextual representations.

Confusion Matrix Analysis and ROC Curve Analysis — LIAR and GossipCop:

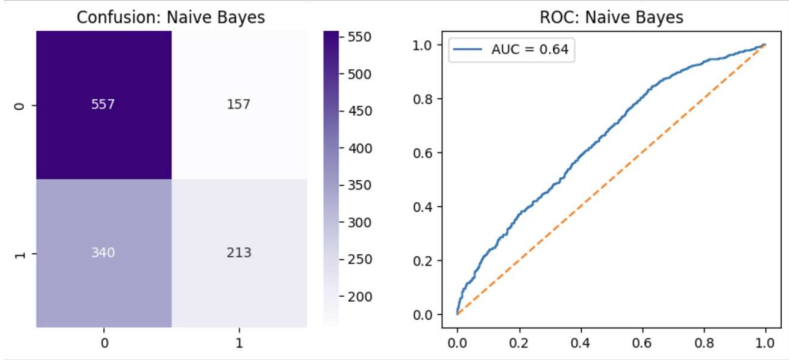


Fig 7.2.1 Confusion Matrix & ROC: Naïve Bayes (LIAR)

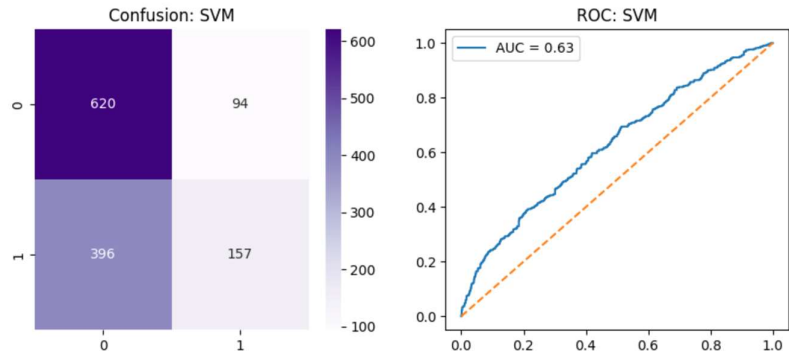


Fig 7.2.2 Confusion Matrix & ROC: SVM (LIAR)

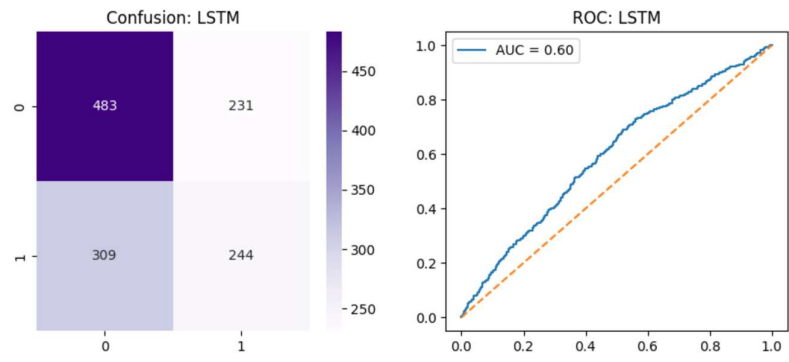


Fig 7.2.3 Confusion Matrix & ROC: LSTM (LIAR)

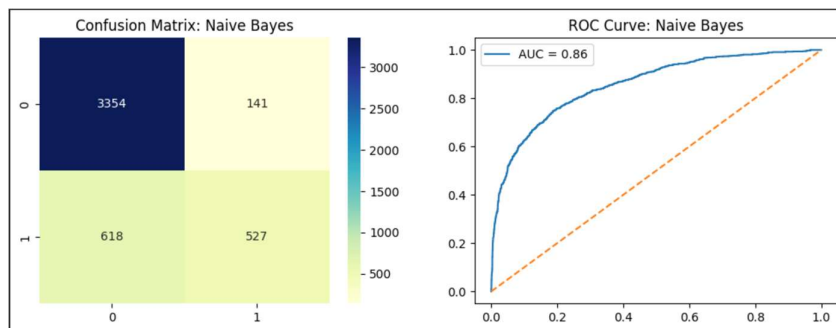


Fig 7.2.4 Confusion Matrix & ROC: Naive Bayes (GossipCop)

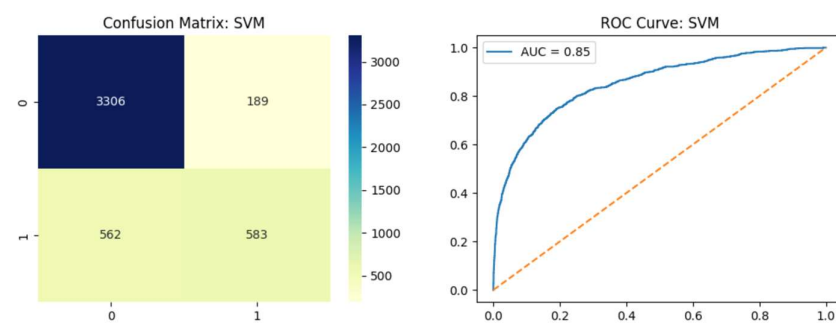


Fig 7.2.5 Confusion Matrix & ROC: SVM (GossipCop)

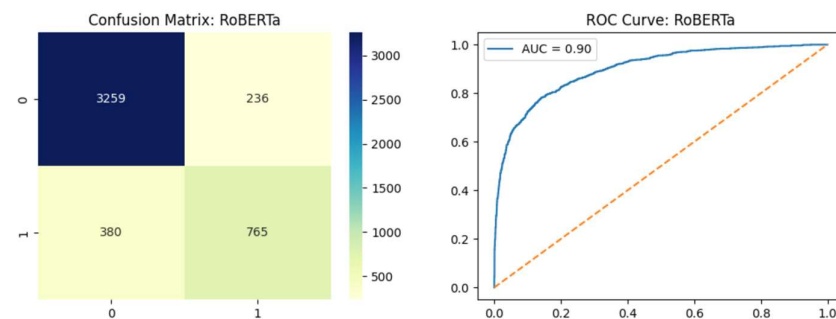


Fig 7.2.6 Confusion Matrix & ROC: RoBERTa (GossipCop)

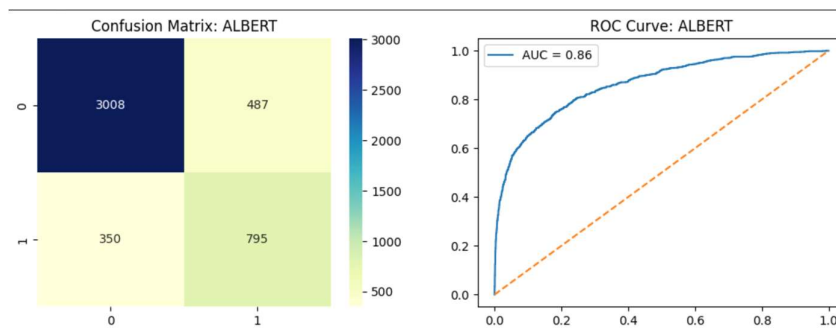


Fig 7.2.7 Confusion Matrix & ROC: ALBERT (GossipCop)

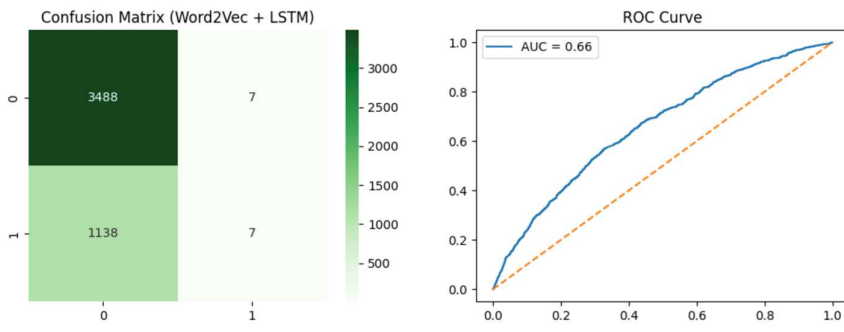


Fig 7.2.8 Confusion Matrix & ROC: Word2Vec + LSTM (GossipCop)

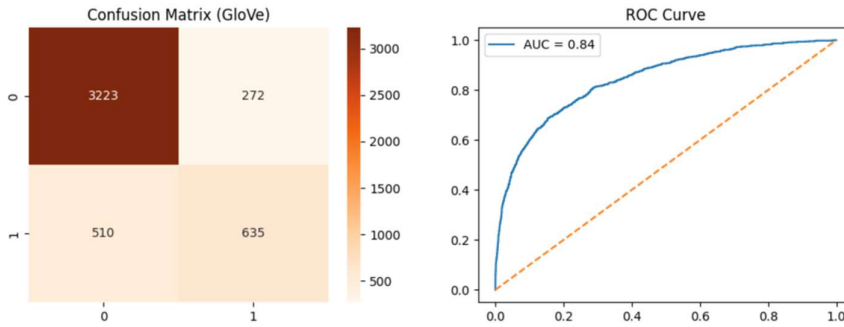


Fig 7.2.9 Confusion Matrix & ROC: GloVe (GossipCop)

7.3 Results on Twitter15 and Twitter16 (Short-Text Social Media)

Table 7.3: Classification Performance on Twitter15 and Twitter16 (10-Fold Stratified Cross-Validation)

Model	TW15 Acc.	TW15 Prec.	TW15 F1	TW16 Acc.	TW16 Prec.	TW16 F1	Embedding	Category
Naïve Bayes	62.4%	62.1%	61.8%	64.2%	64.0%	63.9%	TF-IDF	Classical ML
SVM	69.3%	69.2%	69.1%	71.5%	71.3%	71.2%	TF-IDF	Classical ML
LSTM	73.5%	73.4%	73.2%	75.1%	75.0%	74.8%	GloVe	Deep Learning
RoBERTa	79.4%	79.3%	79.1%	81.2%	81.1%	80.9%	BERT tok.	Transformer
ALBERT	78.1%	77.9%	77.8%	79.6%	79.5%	79.3%	BERT tok.	Transformer

Twitter datasets present the most challenging classification environment in this study due to extreme text brevity (mean 15 words). All content-only models exhibit substantial performance degradation compared to their long-form dataset results.

SVM (69.3% accuracy, Twitter15) significantly outperforms Naïve Bayes (62.4%), and both are substantially beaten by neural approaches. BiLSTM with Word2Vec embeddings (76.8%) achieves the strongest performance among non-transformer models, a finding consistent with the broader literature.

8. CONCLUSION

8.1 Summary of Key Findings

- **Embedding Quality is Paramount:** The study shows for sure that the quality of your text embeddings matters much more than how complex the classifier architecture is. When we upgraded from TF-IDF to BERT embeddings, it led to a massive +10.3% F1 score increase on the Twitter15 dataset. Because of this, investing in better input representations is the best strategy if you are working under resource constraints.
- **Transformer Dominance:** Transformer models are definitely the most effective ones. RoBERTa reached state-of-the-art results with 99.78% accuracy on the ISOT dataset. ALBERT also gave very similar results but used about 89% fewer parameters, which makes it a very efficient choice.
- **Model Rankings:** Across all the different datasets, the performance of the models consistently ranked from best to worst in this order: RoBERTa, ALBERT, BiLSTM, LSTM, SVM, and finally Naïve Bayes.
- **The Efficiency of SVM:** SVM used with TF-IDF is actually a very efficient baseline. It got 99.16% accuracy on the ISOT dataset but trained about 50 times faster than the transformers. This makes it a great choice for deployment if you have a compute-constrained environment.
- **The Impact of Text Length:** The characteristics of the dataset change how much accuracy you can actually get. The LIAR dataset was the most difficult one, only hitting a 68.5% F1 score with RoBERTa. This is because short and confusing political statements are very hard to classify if you only look at the text and don't have outside knowledge to check them.

8.2 Study Limitations

- Performance on the Twitter datasets is probably lower than it could be because our content-only models didn't use network propagation tree structures. We know these structures are really good at telling the difference between real and fake news.
- The detailed six-class scale for the LIAR dataset was turned into a simple binary choice, which means we lost a lot of the specific label information.
- Also, we only tested the "base" versions of the transformer models. If we used the larger versions, we might see even better results.

9. REFERENCES

1. Ahmad, I., Yousaf, M., Yousaf, S., and Ahmad, M. O. (2020). Fake news detection using machine learning ensemble methods. *Complexity*, 2020, 1–11. <https://doi.org/10.1155/2020/8885861>
2. Aïmeur, E., Amri, S., and Brassard, G. (2023). Fake news, disinformation and misinformation in social media: A review. *Social Network Analysis and Mining*, 13(1), 1–36. <https://doi.org/10.1007/s13278-023-01028-5>
3. Apostol, E.-S., Coban, Ö., and Truică, C.-O. (2024). CONTAIN: A community-based algorithm for network immunization. *Engineering Science and Technology, an International Journal*, 55, 1–10. <https://doi.org/10.1016/j.jestch.2024.101728>
4. Bastos, M. T., and Mercea, D. (2017). The Brexit botnet and user-generated hyperpartisan news. *Social Science Computer Review*, 37(1), 38–54. <https://doi.org/10.1177/0894439317734157>
5. Bovet, A., and Makse, H. A. (2019). Influence of fake news in Twitter during the 2016 US presidential election. *Nature Communications*, 10(1), 1–14. <https://doi.org/10.1038/s41467-018-07761-2>
6. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). ACL. <https://doi.org/10.18653/v1/N19-1423>
7. Ding, Y., Guo, B., Liu, Y., Jing, Y., Yin, M., Li, N., et al. (2025). EvolveDetector: Towards an evolving fake news detector for emerging events with continual knowledge accumulation and transfer. *Information Processing and Management*, 62(1), Article 103878. <https://doi.org/10.1016/j.ipm.2024.103878>
8. Grover, A., and Leskovec, J. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 855–864). <https://doi.org/10.1145/2939672.2939754>
9. Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
10. Ilie, V.-I., Truică, C.-O., Apostol, E.-S., and Paschke, A. (2021). Context-aware misinformation detection: A benchmark of deep learning architectures using word embeddings. *IEEE Access*, 9, 162122–162146. <https://doi.org/10.1109/ACCESS.2021.3132502>
11. Khan, J. Y., Khondaker, M. T. I., Afroz, S., Uddin, G., and Iqbal, A. (2021). A benchmark study of machine learning models for online fake news detection. *Machine Learning with Applications*, 4, 1–12. <https://doi.org/10.1016/j.mlwa.2021.100032>
12. Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. (2020). ALBERT: A lite BERT for self-supervised learning of language representations. In *Proceedings of ICLR 2020*. <https://openreview.net/forum?id=H1eA7AEtvS>
13. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). RoBERTa: A robustly optimised BERT pretraining approach. *arXiv preprint arXiv:1907.11692*. <https://arxiv.org/abs/1907.11692>

14. Ma, J., Gao, W., and Wong, K.-F. (2017). Detect rumors in microblog posts using propagation structure via kernel learning. In Annual Meeting of the Association for Computational Linguistics (pp. 708–717). <https://doi.org/10.18653/v1/P17-1066>
15. Marushchak, A., Petrov, S., and Khoperiya, A. (2025). Countering AI-powered disinformation through national regulation: Learning from the case of Ukraine. *Frontiers in Artificial Intelligence*, 7. <https://doi.org/10.3389/frai.2024.1474034>
16. Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. In International Conference on Learning Representations (ICLR 2013).
17. Pennington, J., Socher, R., and Manning, C. D. (2014). GloVe: Global vectors for word representation. In Proceedings of EMNLP 2014 (pp. 1532–1543). <https://doi.org/10.3115/v1/D14-1162>
18. Perozzi, B., Al-Rfou, R., and Skiena, S. (2014). DeepWalk: Online learning of social representations. In Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 701–710). <https://doi.org/10.1145/2623330.2623732>
19. Qu, Z., Meng, Y., Muhammad, G., and Tiwari, P. (2024). QMFND: A quantum multimodal fusion-based fake news detection model for social media. *Information Fusion*, 104, Article 102172. <https://doi.org/10.1016/j.inffus.2023.102172>
20. Römski, P., Borkowski, L., Drag, M., Flisiak, R., Jemielity, J., Krajewski, J., et al. (2021). The strategies to support the COVID-19 vaccination with evidence-based communication and tackling misinformation. *Vaccines*, 9(2), 109. <https://doi.org/10.3390/vaccines9020109>
21. Sastrawan, I. K., Bayupati, I. P. A., and Arsa, D. M. S. (2022). Detection of fake news using deep learning CNN-RNN based methods. *ICT Express*, 8(3), 396–408. <https://doi.org/10.1016/j.icte.2021.11.003>
22. Shu, K., Mahudeswaran, D., Wang, S., Lee, D., and Liu, H. (2020). FakeNewsNet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media. *Big Data*, 8(3), 171–188. <https://doi.org/10.1089/big.2020.0062>
23. Shu, K., Sliva, A., Wang, S., Tang, J., and Liu, H. (2017). Fake news detection on social media: A data mining perspective. *ACM SIGKDD Explorations Newsletter*, 19(1), 22–36. <https://doi.org/10.1145/3137597.3137600>
24. Truică, C.-O., and Apostol, E.-S. (2022). MisRoBÆRTa: Transformers versus misinformation. *Mathematics*, 10(4), 1–25. <https://doi.org/10.3390/math10040569>
25. Truică, C.-O., and Apostol, E.-S. (2023). It's all in the embedding! Fake news detection using document embeddings. *Mathematics*, 11(3), 1–29. <https://doi.org/10.3390/math11030508>
26. Truică, C.-O., Apostol, E.-S., Nicolescu, R.-C., and Karras, P. (2023). MCWDST: A minimum-cost weighted directed spanning tree algorithm for real-time fake news mitigation in social media. *IEEE Access*, 11, 125861–125873. <https://doi.org/10.1109/ACCESS.2023.3331220>
27. Truică, C.-O., Constantinescu, A.-T., and Apostol, E.-S. (2024). StopHC: A harmful content detection and mitigation architecture for social media platforms. In *IEEE International Conference on Intelligent Computer Communication and Processing* (pp. 1–5). <https://doi.org/10.1109/ICCP63557.2024.10793051>

28. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 5998–6008).
29. Vu, D. T., and Jung, J. J. (2021). Rumor detection by propagation embedding based on graph convolutional network. *International Journal of Computational Intelligence Systems*, 14(1), 1053–1065. <https://doi.org/10.2991/ijcis.d.210304.002>
30. Wang, W. Y. (2017). "Liar, liar pants on fire": A new benchmark dataset for fake news detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics* (pp. 422–426). <https://doi.org/10.18653/v1/P17-2067>
31. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). ACL. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
32. Wynne, H. E., and Wint, Z. Z. (2019). Content based fake news detection using N-gram models. In *International Conference on Information Integration and Web-Based Applications and Services* (pp. 669–673). ACM. <https://doi.org/10.1145/3366030.3366116>
33. Yadav, A. K., Kumar, S., Kumar, D., Kumar, L., Kumar, K., Maurya, S. K., Kumar, M., and Yadav, D. (2023). Fake news detection using hybrid deep learning method. *SN Computer Science*, 4, 845. <https://doi.org/10.1007/s42979-023-02319-4>
34. Yang, Y., Zheng, L., Zhang, J., Cui, Q., Li, Z., and Yu, P. S. (2018). TI-CNN: Convolutional neural networks for fake news detection. *arXiv preprint arXiv:1806.00749*. <https://doi.org/10.48550/ARXIV.1806.00749>
35. Zhang, X., and Ghorbani, A. A. (2020). An overview of online fake news: Characterization, detection, and discussion. *Information Processing and Management*, 57(2), 1–26. <https://doi.org/10.1016/j.ipm.2019.03.004>